

Conversão de Modelo ER em Modelo Relacional

Contexto

Uma empresa deseja desenvolver um sistema de streaming musical semelhante ao Spotify. O sistema permitirá que usuários escutem músicas, sigam artistas, criem playlists, curtam faixas e acessem álbuns publicados por artistas.

Sua tarefa é modelar esse sistema em duas etapas:

1. Criar um **Diagrama Entidade-Relacionamento (DER)**.
2. Transformar o DER em um **modelo relacional**.

Descrição do sistema

O sistema deve armazenar informações sobre os usuários, incluindo nome, e-mail, data de nascimento e tipo de plano assinado. Cada usuário pode possuir um plano gratuito ou premium. Um plano possui nome, valor mensal e limite de recursos.

Os usuários podem criar playlists. Cada playlist possui título, data de criação e uma indicação se ela é pública ou privada. Uma playlist pertence a apenas um usuário, mas um usuário pode criar várias playlists.

As playlists são compostas por músicas. Uma música pode aparecer em várias playlists, e uma playlist pode conter várias músicas. Para cada música adicionada a uma playlist, deve-se registrar a data em que ela foi adicionada e a posição da música dentro da playlist.

Cada música possui título, duração, data de lançamento e número de reproduções. Uma música pertence a um álbum, mas um álbum pode conter várias músicas. Um álbum possui título, data de lançamento e tipo, que pode ser single, EP ou álbum completo.

Os álbuns são publicados por artistas. Um artista possui nome artístico, país de origem e data de início da carreira. Um artista pode publicar vários álbuns.

Além disso, uma música pode ter a participação de mais de um artista, como ocorre em colaborações musicais. Nesse caso, deve-se registrar o papel do artista na música, por exemplo: principal, participação especial ou produtor.

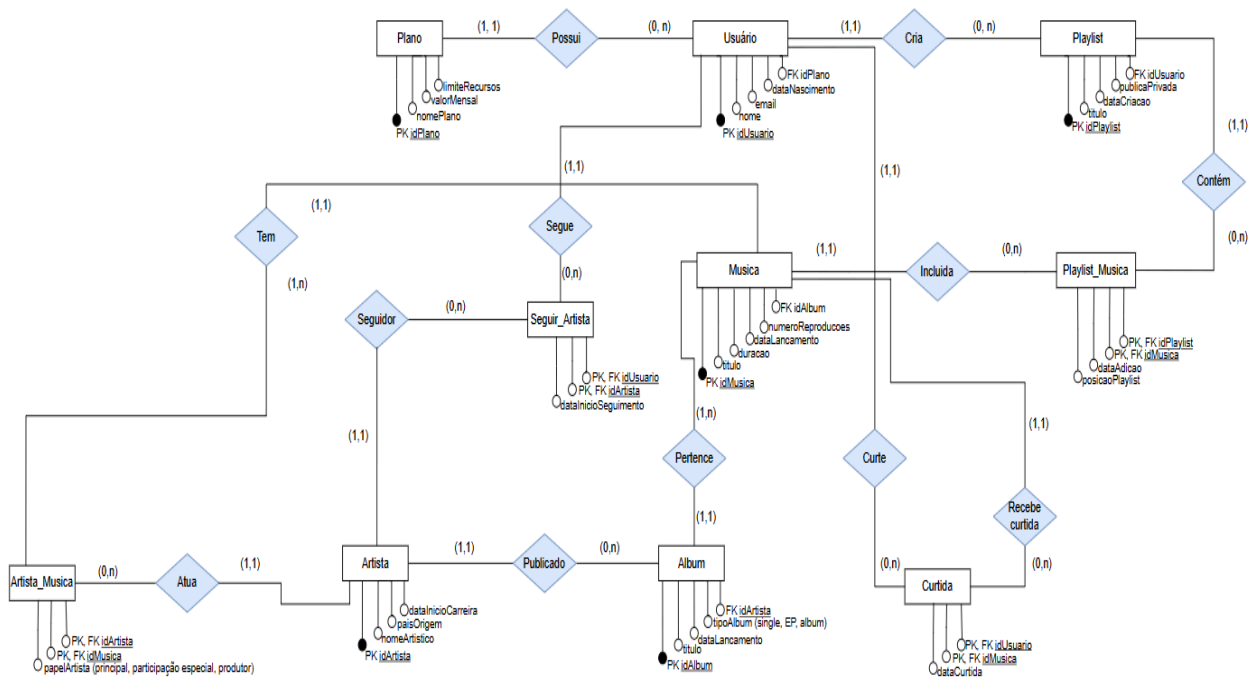
Os usuários também podem seguir artistas. Um usuário pode seguir vários artistas, e um artista pode ser seguido por vários usuários. Para cada relação de acompanhamento, deve-se armazenar a data em que o usuário começou a seguir o artista.

Além disso, os usuários podem curtir músicas. Um usuário pode curtir várias músicas, e uma música pode ser curtida por vários usuários. Para cada curtida, deve-se registrar a data da curtida.

Parte 1 - Construção do DER

Com base na descrição anterior, construa um **Diagrama Entidade-Relacionamento** contento:

- As entidades do sistema;
- Os atributos de cada entidade;
- As chaves primárias de cada entidade;
- Os relacionamentos entre as entidades;
- As cardinalidades mínimas e máximas dos relacionamentos;
- Os atributos de relacionamentos, quando existirem;
- A identificação de relacionamentos muito-para-muitos;
- A indicação de entidades associativas, quando necessário.



Parte 2 - Transformação para o Modelo Relacional

Transforme o DER criado na Parte 1 em um **modelo relacional**.

Para cada tabela, indique:

- Nome da tabela;
- Atributos da tabela;
- Chave primária;
- Chaves estrangeiras;

e) Restrições importantes, quando existirem.

```
PLANO(  
    idPlano PK,  
    nomePlano,  
    valorMensal,  
    limiteRecursos  
)
```

```
USUARIO(  
    idUsuario PK,  
    nome,  
    email,  
    dataNascimento,  
    idPlano FK → PLANO(idPlano)  
)
```

```
ARTISTA(  
    idArtista PK,  
    nomeArtistico,  
    paisOrigem,  
    dataInicioCarreira  
)
```

```
ALBUM(  
    idAlbum PK,  
    titulo,  
    tipoAlbum,  
    dataLancamento,  
    idArtista FK → ARTISTA(idArtista)  
)
```

```
MUSICA(  
    idMusica PK,  
    titulo,  
    duracao,  
    dataLancamento,  
    numeroReproducoes,  
    idAlbum FK → ALBUM(idAlbum)  
)
```

```
PLAYLIST(  
    idPlaylist PK,  
    titulo,  
    dataCriacao,  
    publicaPrivada,  
    idUsuario FK → USUARIO(idUsuario)  
)
```

```
PLAYLIST_MUSICA(  
    idPlaylist PK FK → PLAYLIST(idPlaylist),  
    idMusica PK FK → MUSICA(idMusica),  
    dataAdicao,  
    posicaoPlaylist  
)
```

```
CURTIDA(  
    idUsuario PK FK → USUARIO(idUsuario),  
    idMusica PK FK → MUSICA(idMusica),  
    dataCurtida  
)
```

```
SEGUIR_ARTISTA(  
    idUsuario PK FK → USUARIO(idUsuario),  
    idArtista PK FK → ARTISTA(idArtista),  
    papelArtista  
)
```

Regras mínimas que o modelo deve atender

O modelo criado deve representar, obrigatoriamente:

1. Usuários e seus planos;
2. Playlists criadas por usuários;
3. Músicas adicionadas a playlists;
4. Álbuns compostos por músicas;
5. Artistas responsáveis por álbuns;
6. Participação de artistas em músicas;
7. Usuários seguindo artistas;
8. Usuários curtindo músicas.

Questões para reflexão

Após criar o modelo, responda:

1. Quais relacionamentos do sistema são muitos-para-muitos?
 - Playlist ↔ Música
 - Usuário ↔ Música (curtidas)
 - Usuário ↔ Artista (seguimento)
 - Artista ↔ Música
2. Quais tabelas surgiram a partir da transformação desses relacionamentos?
 - Playlist_Musica
 - Curtida
 - Seguir_Artista
 - Artista_Musica

3. Por que a relação entre playlist e música não deve ser representada apenas como uma chave estrangeira em música?

Porque uma música pode estar em várias playlists diferentes ao mesmo tempo. Se existisse apenas uma FK em Música apontando para Playlist, cada música poderia pertencer somente a uma única playlist, o que não representa corretamente o funcionamento real de um sistema de streaming. Além disso, a relação possui atributos próprios:

- dataAdição
- posicaoPlaylist

Por isso é necessária a tabela intermediária Playlist_Musica.

4. Por que a relação entre usuário e música curtida precisa de uma tabela própria?

Porque:

- Um usuário pode curtir várias músicas;
- Uma música pode ser curtida por vários usuários.

Isso caracteriza um relacionamento muitos-para-muitos.

Além disso, a relação possui o atributo:

- dataCurtida

Portanto, é necessário criar a tabela Curtida.

5. Qual seria o impacto de armazenar todas as músicas de uma playlist em um único atributo textual?

- Violação da 1ª Forma Normal (1FN);
- Dificuldade para buscar músicas específicas;
- Dificuldade para remover ou adicionar músicas;
- Redundância de dados;
- Pior desempenho em consultas;
- Impossibilidade de usar integridade referencial com chaves estrangeiras.

O correto é armazenar cada música separadamente na tabela Playlist_Musica.

6. Em quais tabelas aparecem chaves estrangeiras? O que cada uma delas representa?

Tabela	Chave Estrangeira	Representa
Usuario	idPlano	plano do usuário
Album	idArtista	artista que publicou o álbum
Musica	idAlbum	álbum ao qual a música pertence
Playlist	idUsuario	usuário que criou a playlist
Playlist_Musica	idPlaylist	playlist relacionada
Playlist_Musica	idMusica	música adicionada

Curtida	idUsuario	usuário que curtiu
Curtida	idMusica	música curtida
Seguir_Artista	idUsuario	usuário seguidor
Seguir_Artista	idArtista	artista seguido
Artista_Musica	idArtista	artista participante
Artista_Musica	idMusica	música relacionada

Faça uma breve justificativa das principais decisões de modelagem.

O modelo foi construído para representar corretamente as regras de negócio de uma plataforma de streaming musical. As principais decisões foram:

1. Utilização de tabelas associativas para relacionamentos muitos-para-muitos;
2. Uso de chaves primárias compostas nas tabelas associativas;
3. Aplicação de chaves estrangeiras para garantir integridade referencial;
4. Separação de entidades principais (Usuário, Música, Álbum, Artista, Playlist e Plano);
5. Armazenamento de atributos específicos do relacionamento nas tabelas intermediárias.

Essa modelagem reduz redundância, facilita consultas e mantém o banco de dados consistente e normalizado.

